Week 7 - Finding the k-th Smallest Element with a Heap

Problem

Given an unsorted array of `n` integers and an integer `k`, find the k-th smallest value.

For `array = [7, 10, 4, 3, 20, 15]` and `k = 3`, the answer is `7`.

Naive solution: sort the whole array

Sort the array, then return the value at index `k - 1`.

```
[7, 10, 4, 3, 20, 15]
→ [3, 4, 7, 10, 15, 20]
          ^
     3rd smallest
```

This takes `O(n log n)` time.

Another way to think about it

The k-th smallest value is the **largest of the k smallest values**. Instead of sorting every value, keep track of only those `k` smallest candidates.

Key idea

Keep the `k` smallest values seen so far in a max heap.

1. Put the first `k` array values into the heap.
2. For each remaining value, compare it with the heap root.
3. If it is smaller than the root, replace the root with that value.
4. After processing the array, the root is the k-th smallest value.

C++ implementation

```cpp
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int kthSmallest(const vector<int>& arr, int k) {

    priority_queue<int> maxHeap;
    // priority_queue< int, vector<int>, greater<int> > minHeap;

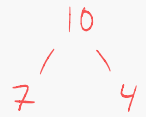
    // Put the first k values into the max heap.
    for (int i = 0; i < k; ++i) {
        maxHeap.push(arr[i]);
    }

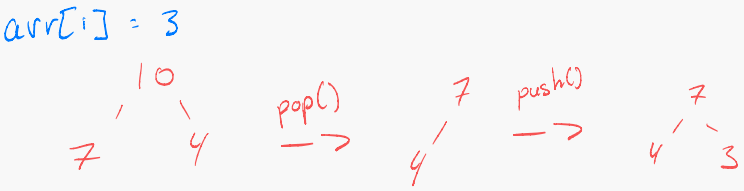
    // Keep only the k smallest values seen so far.
    for (int i = k; i < arr.size(); ++i) {
        if (arr[i] < maxHeap.top()) {
            maxHeap.pop();
            maxHeap.push(arr[i]);
        }
    }

    return maxHeap.top();
}

int main() {
    vector<int> arr = {7, 10, 4, 3, 20, 15};
    int k = 3;

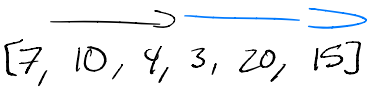
    cout << "k-th smallest item: " << kthSmallest(arr, k) << endl;
    return 0;
}
```





Walkthrough

Use `[7, 10, 4, 3, 20, 15]` with `k = 3`.



1. Insert `7`, `10`, and `4`. The max heap is `{10, 7, 4}`.
2. Read `3`. Since `3 < 10`, replace `10`; the heap becomes `{7, 4, 3}`.
3. Read `20` and `15`. Both are larger than the root (`7`), so ignore them.
4. The root is `7`, the third smallest value.



Complexity

- **Time:** $O(n \log k)$
- **Extra space:** $O(k)$

How much work does this save?

Suppose $n = 1{,}000{,}000$ and $k = 10$.

- Sorting needs roughly $n \log_2 n = 1{,}000{,}000 \times 20$ = **20 million** units of sorting work.
- The heap approach needs roughly $n \log_2 k = 1{,}000{,}000 \times 3.3$ = **3.3 million** units of heap work.

When k is small, that is about **6 times less work** than sorting the entire array.